Operadic PFP: Typed Composition and Bounded DDC Repair

Jeremy H. Carroll

2026-10-21

Version 0.1.0

Code baseline: 6d9da6fa7919ae2a8b99b704e784e5a4f422aed2

Abstract

Proposal Fidelity Protocol separates a candidate from the checks performed on it and from permission to use it. This implementation study describes the current operadic organization of that separation and one executable, provider-free code-repair example. Capability definitions and typed composition belong to Omega Engine Operads (OEOP); execution belongs to Woland; immune observations belong to OEEIS; evolution and lineage belong to Prompt Civilization. A supplied patch passes through six source-bound operations on an isolated working copy. The example repairs one branch while preserving one negative control. Its result is finite test evidence, not a theorem about the program, a production admission or a measurement of model performance.

1. Introduction

An executable proposal protocol needs more than a prompt and a favorable status label. It needs a declared task, identifiable candidate bytes, independent checks and an account of what actually ran. Earlier papers in this series distinguish task-bound claims from surrounding prose [1] and local compatibility from a global result [2]. Here those distinctions are applied to the existing typed compiler and an effectful local repair workflow.

OEOP is the renamed capability and composition subsystem formerly called OEWAS. The rename does not rename historical protocol identifiers or convert every discovered skill into an admitted executable. The contribution is an implemented organization and a bounded worked example. No universal claim about operads, model reasoning or arbitrary software repair is made.

2. Primitives, carriers and ownership

A repair request fixes an objective, a manifest of source-file hashes, an allowed write set, a supplied text patch and a tuple of check commands. Its candidate identity hashes the patch bytes; a separate digest binds the entire request. These identities are not interchangeable: the same patch can be tested on different sources or under different checks.

An operation has ordered input and output slots, declared effects, source bindings and an executor binding. A compiled workflow records their composition. An execution additionally records callbacks and observations. Typed composition constrains connections; it does not prove semantic correctness of a callback.

Owner	Responsibility	Boundary
OEOP	Capability sources, typed declarations and composition	Discovery and successful compilation do not admit execution.

Owner	Responsibility	Boundary
Woland	Execution, working copies, checks and retained run evidence	Local check commands are trusted configuration, not an operating-system sandbox.
OEEIS	Immune definitions, screening and activation decisions	A missing measurement is not clearance or activation.
Prompt Civilization	Evolution records, genealogy and outcome lineage	A lineage record is not a proof or a release authorization.

Editable Markdown capability packages, workflows and shared configuration have OEOP sources. Antibody and anti-skill definitions have OEEIS sources. The `.agents`, `.claude`, `.codex` and `.grok` copies remain ordinary generated discovery files. Host-specific configuration is retained; runtime journals and leases are not new instruction authorities. This source split does not merge the four subsystems into one runtime.

3. Composition contract and execution claims

The implemented repair slice uses the ordered trace C, R, E, I, M, T. These six operations use existing typed workflow, operation-inventory and native execution boundaries. They do not implement every move of the broader Discrete Deformation Calculus: no representation lift or mathematical descent law is inferred from the six-letter trace.

1. **C — capture.** Check the declared source hashes, retain the request and patch, and create a fresh working copy outside the original source tree.
2. **R — observe.** Run the fixed check commands to obtain baseline observations.
3. **E — evaluate.** Check patch paths against the allowed existing files, apply the patch in the working copy, and repeat the same checks.
4. **I — incorporate.** Submit bound observations through the existing immune and genealogy interfaces. Retain missing-service or binding failures.
5. **M — disposition.** Compare the declared baseline and post-patch outcomes and distinguish repair, partial improvement, ineffectiveness, regression, refusal and a baseline requiring no repair.
6. **T — terminal.** Retain the execution record. Source application is not performed; production admission and detector activation are not granted.

The state carries the run key, request digest and candidate identity through the sequence. Missing executor bindings refuse before operations take effect. Unavailable native compilation does not produce an execution receipt. A `REPAIRED` result concerns the supplied checks only. Neither type compatibility nor a matching hash proves that the chosen checks express the desired task.

The execution record and the optional learning notification are separate. Failure of a later notification cannot replace retained observations with a fabricated successful learning result. These are executable boundary choices, not additional development commit gates.

4. Worked example: one positive branch and one control

The source defines `adjust(n)` to return `n` for a negative input and `n - 1` otherwise. The declared positive check requires `adjust(2) == 3`; the control requires `adjust(-2) == -2`. The supplied patch changes only the nonnegative branch from `n - 1` to `n + 1`.

Before the patch, check exits are `[1, 0]`; afterward they are `[0, 0]`. The native workflow coordinator records six operation steps; working-copy and process effects execute in Python infrastructure callbacks. The run retains the working copy and observations. The original source file remains byte-identical. The example does not test every integer, discover the patch, call a model or install the repaired program.

This is deliberately a small demonstration. It establishes the exercised capture, patch, check and evidence path. Replacing these two checks with weak checks could admit an inappropriate patch. Repeating the example with a new run identifier is another local replay, not another independent model trial.

5. Mechanization and verification

The implementation is recorded in the source set listed in the companion provenance and reproduction note. The existing DDC test target exercises successful repair, ineffective and regressive candidates, source drift, forbidden patch scope, missing execution infrastructure and integration binding failures. It also tests inherited Git addressing and same-size Python edits, which can otherwise reuse stale bytecode.

The check profile disables Python bytecode writes and clears inherited optimization and cache-prefix overrides. It records output-prefix truncation explicitly. Hashes of captured output prefixes must not be read as hashes of an unbounded complete stream. Per-command deadlines and retained output limits are engineering controls, not a proof of total resource confinement.

The worked example uses the real OEEIS candidate-checking interface and the real genealogy interface. An observed `NOT_MEASURED` immune result is retained as such; activation stays on hold. The terminal record reports mathematical measurement as not performed. No new Lean theorem or axiom census is claimed. Where an existing paper kernel is rebuilt, the active estate remains on the single Lean v4.35.0 version; the engineering example does not add another pin.

6. Discussion and non-claims

This workflow is a supplied-candidate executor, not an autonomous code-repair agent. It makes no provider call and reports no improvement in model accuracy, cost or safety. Its check commands are trusted host input and may execute arbitrary local code; isolating a working copy is not a security sandbox.

The example does not establish general program correctness, mathematical convergence, arbitrary prose equivalence or complete implementation of DDC. It does not apply source changes, publish a package, activate a detector or authorize an external action. Skill discovery, compilation, observed execution, mathematical proof and authorization remain separate evidence categories.

Future work should start from a concrete failed task or missing integration, retain the failing candidate and a benign control, and repair the existing owner. A larger catalog, more generated guidance or a longer trace is not by itself evidence of better proposals. Matched model comparisons, when separately authorized, require their own raw responses, checker bindings and cost records.

References

[1] Jeremy H. Carroll. Task-bound claims and the scope of hallucination control, version 0.2.0, author-review draft, 2026-10-21.

[2] Jeremy H. Carroll. Local witnesses and global compatibility in agent composition, version 0.2.0, author-review draft, 2026-10-21.

[3] Source provenance and executable reproduction. This supplies implementation evidence, not an external publication identifier.